NetworkPlanningTooV3 Linux 服务器版本部署指南

文档版本：v1.1 — 2026-05-21
目标系统：Armbian Linux (ARM64/aarch64)
部署方式：FastAPI 后端 + Cloudflare Tunnel 内网穿透 + Tailscale Funnel 备用通道

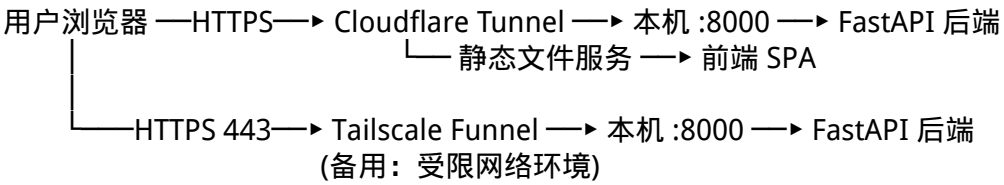
目录

1. 环境概述
2. 后端部署
3. 前端构建与部署
4. Cloudflare Tunnel 内网穿透
   - 4.6 Tailscale Funnel 备用通道
5. Systemd 开机自启服务
6. 域名访问验证
7. 常见问题与故障排除
   - 7.4 Cloudflare 隧道连接异常 / 部分网络无法访问
8. 维护命令速查
   - 8.4 Tailscale Funnel 备用通道
   - 8.6 一键全部重启

---

1. 环境概述

1.1 项目架构



1.2 环境信息

项目	值
操作系统	Armbian Linux (RK3588 / ARM64)
后端框架	FastAPI (Python 3)
前端框架	React + Vite + TypeScript
主内网穿透	Cloudflare Tunnel (cloudflared, HTTP2 协议)
备用内网穿透	Tailscale Funnel (HTTPS 443, 突破企业防火墙)
域名（主）	app.pengwc.asia
域名（备）	armbian.tail362392.ts.net
后端地址	http://192.168.101.168:8000
Cloudflare 端口	TCP 7844（非标准端口，部分企业防火墙阻断）
Tailscale Funnel 端口	TCP 443（标准 HTTPS，几乎所有防火墙放行）

1.3 前置条件

- Python 3.8+（后端运行环境）
- Node.js 18+（前端构建环境）
- Cloudflare 账号及已配置域名
- 已安装 cloudflared（Cloudflare Tunnel 客户端）

---

2. 后端部署

2.1 创建虚拟环境并安装依赖

```
cd /DATA/AppData/NetworkPlanningTooV3/backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install uvicorn fastapi pandas openpyxl  # 补充核心依赖
```

2.2 启动后端服务

开发模式（热重载）：

```
cd /DATA/AppData/NetworkPlanningTooV3/backend
python3 main.py
```

生产模式（推荐用于 systemd 服务）：

```
cd /DATA/AppData/NetworkPlanningTooV3/backend
python3 -m uvicorn main:app --host 0.0.0.0 --port 8000
```

2.3 验证后端

```
# 健康检查
curl http://127.0.0.1:8000/health
# 预期输出: {"status":"healthy"}

# 系统信息
curl http://127.0.0.1:8000/api/v1/system/info
# 预期输出: {"version":"1.0.0","backend_ip":"192.168.101.168","backend_port":8000}

# API 文档
curl -o /dev/null -w "%{http_code}" http://127.0.0.1:8000/docs
# 预期输出: 200
```

2.4 查看可用 API 路由

```
curl -s http://127.0.0.1:8000/openapi.json | python3 -c \
  "import json,sys; [print(p) for p in sorted(json.load(sys.stdin).get('paths',{}).keys())]"
```

---

3. 前端构建与部署

3.1 核心策略：后端托管前端静态文件

本项目采用 FastAPI 后端直接托管前端 SPA 静态文件 的策略，而非单独运行前端服务器。

- 修改 backend/app/api/__init__.py，添加静态文件路由
- 前端构建产物输出到 frontend/dist-renderer/
- 所有请求通过同一个端口（8000）处理，API 路由优先匹配，其余路径回退到 SPA

3.2 后端代码修改

backend/app/api/__init__.py 中需要添加的核心配置：

```
from pathlib import Path
from fastapi.responses import FileResponse, HTMLResponse

# 前端构建产物目录
FRONTEND_DIST = Path(__file__).parent.parent.parent.parent / "frontend" / "dist-renderer"

# 在 create_app() 函数中：

# 修改根路由：构建了前端则返回 HTML
@app.get("/")
async def root():
    index_path = FRONTEND_DIST / "index.html"
    if index_path.exists():
        return HTMLResponse(index_path.read_text(encoding="utf-8"))
    return {"message": "网络规划工具API", "version": settings.VERSION}

# SPA 回退路由（必须放在所有路由最后）
@app.get("/{full_path:path}")
async def serve_frontend(full_path: str):
    if full_path.startswith("api/") or full_path.startswith("static/") \
       or full_path == "health":
        raise HTTPException(status_code=404)

    file_path = FRONTEND_DIST / full_path
    if file_path.exists() and file_path.is_file():
        return FileResponse(file_path)

    # SPA 回退：所有前端路由返回 index.html
    index_path = FRONTEND_DIST / "index.html"
    if index_path.exists():
        return HTMLResponse(index_path.read_text(encoding="utf-8"),
                            media_type="text/html")

    raise HTTPException(status_code=404)
```

重要： CORS 配置中需允许域名访问：

```
allow_origin_regex=r"(https://.*\.pengwc\.as)|..."
```

3.3 构建前端

3.3.1 安装依赖

```
cd /DATA/AppData/NetworkPlanningTooV3/frontend
npm install
```

3.3.2 ARM64 架构兼容性处理　在 ARM 架构（如 RK3588）上，node_modules 可能缺少平台原生模块。关键步骤：

```
# 重新安装以获取 ARM64 原生模块
cd /DATA/AppData/NetworkPlanningTooV3/frontend
npm install  # 会替换 x86 原生包为 ARM64 版本
```

3.3.3 处理缺失依赖的问题　源码可能引用了 package.json 中未声明的依赖（历史遗留问题）。典型缺失包：

```
npm install i18next react-i18next    # 国际化
npm install react-colorful          # 颜色选择器
npm install terser                  # 代码压缩（vite 配置需要）
```

3.3.4 执行构建

```
cd /DATA/AppData/NetworkPlanningTooV3/frontend

# 如果 vite 无执行权限
chmod +x node_modules/.bin/vite

# 构建
npm run build:vite
# 或
npx vite build
```

构建产物输出到 frontend/dist-renderer/ 目录。

3.3.5 验证构建产物

```
ls -la /DATA/AppData/NetworkPlanningTooV3/frontend/dist-renderer/
# 应包含: index.html + assets/ 目录（含 JS/CSS 文件）

# 检查 JS 文件是否有正确的哈希（文件重名说明是新构建）
ls -la /DATA/AppData/NetworkPlanningTooV3/frontend/dist-renderer/assets/js/
```

判断构建是否最新的依据：

对比项	旧（需重新构建）	新（已更新）
JS 文件名	index-DJRc6RDA.js	index-xxxxxxx.js（哈希变化）
文件时间	早于源码修改时间	等于或晚于源码修改时间
文件大小	较小（旧代码少）	较大（新代码增加）

3.4 前端服务说明

重要区别：

访问模式	说明	适用场景
后端托管模式	FastAPI 直接服务静态文件	生产部署（当前采用）
Vite 开发服务器	npm run dev:web，端口 5173	开发调试阶段
Electron 桌面模式	npm run dev	桌面应用开发

---

4. Cloudflare Tunnel 内网穿透

4.1 安装 cloudflared

```
# ARM64 版本
wget https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm64
mv cloudflared-linux-arm64 /usr/local/bin/cloudflared
chmod +x /usr/local/bin/cloudflared
```

4.2 配置隧道

在 Cloudflare Zero Trust Dashboard 中创建隧道，获取 token，然后：

```
cloudflared tunnel run --token <YOUR_TOKEN>
```

4.3 Systemd 服务（cloudflared）

创建 /etc/systemd/system/cloudflared.service：

```
[Unit]
Description=cloudflared
After=network-online.target
Wants=network-online.target

[Service]
TimeoutStartSec=30
Type=notify
ExecStart=/usr/bin/cloudflared --protocol http2 --edge-ip-version auto --no-autoupdate tunnel run --token <YOUR_
Restart=always
RestartSec=10s

[Install]
WantedBy=multi-user.target
```

关键参数说明：

参数	说明
--protocol http2	必须使用 HTTP2 (TCP) 协议。默认 QUIC (UDP) 在部分网络上超时严重

参数	说明
--edge-ip-version auto	自动选择 IPv4/IPv6
TimeoutStartSec=30	启动超时设为 30 秒（默认 15 秒不够）
Type=notify	cloudflared 就绪后通知 systemd

启用服务：

```
systemctl daemon-reload
systemctl enable cloudflared
systemctl start cloudflared
```

4.4 验证隧道状态

```
systemctl status cloudflared --no-pager

# 查看连接日志（确认 protocol=http2）
journalctl -u cloudflared -n 20 --no-pager | grep "Registered tunnel connection"

# 预期输出：
# Registered tunnel connection ... location=hkg12 protocol=http2
# Registered tunnel connection ... location=lax08 protocol=http2
```

4.5 隧道配置要点

- Cloudflare Dashboard 中的隧道目标地址指向本机 http://192.168.101.168:8000
- Cloudflare 会自动管理 SSL 证书，用户通过 HTTPS 访问
- Cloudflare Tunnel 连接目标是 TCP 7844 端口（非标准端口）
  - 家庭 WiFi / 手机数据网络通常放行 → 可正常访问
  - 企业/办公室防火墙通常只放行 80/443 → 端口 7844 被阻断，无法访问
  - 受限网络下请使用备用通道：Tailscale Funnel（见 4.6 节）
- Cloudflare 不会缓存 HTML 类型响应（cf-cache-status: DYNAMIC），确保前端始终最新
- 静态资源（JS/CSS）的缓存时间由后端返回的 cache-control 头控制

4.6 Tailscale Funnel 备用通道（突破企业防火墙）

为什么需要备用通道： Cloudflare Tunnel 使用 TCP 7844 端口，企业/办公室防火墙通常只放行 80/443，导致通过 app.pengwc.asia 无法访问。Tailscale Funnel 走标准 HTTPS 443 端口，几乎任何能上网的网络都可

4.6.1 前置条件

```
# 安装 Tailscale（如未安装）
curl -fsSL https://tailscale.com/install.sh | sh

# 启动并登录
tailscale up
```

重要： 需要在 Tailscale 管理后台（https://login.tailscale.com/admin/dns）启用 HTTPS 证书和 Funnel 功能。

4.6.2 配置 Funnel

```
# 第 1 步：配置反向代理规则
tailscale serve --bg --set-path / http://localhost:8000

# 第 2 步：开启 Funnel（暴露到公网）
tailscale funnel --bg --set-path / http://localhost:8000

# 第 3 步：验证
tailscale funnel status
# 输出: https://armbian.tail362392.ts.net (Funnel on)
#       |-- / proxy http://localhost:8000
```

4.6.3 验证 Funnel 公网访问

```
curl -s -o /dev/null -w "%{http_code}" https://armbian.tail362392.ts.net/health
# 预期: 200
```

4.6.4 Funnel 特性

特性	说明
访问地址	https://armbian.tail362392.ts.net/
端口	TCP 443（标准 HTTPS）
是否需要客户端	否，任何浏览器直接打开
开机自启	□ 配置存储在 Tailscale 云端，随 tailscaled 服务自动恢复
适用网络	所有能访问互联网的网络（含企业/办公室防火墙环境）

4.6.5 双通道架构总结

普通网络（家庭/手机）──▸ app.pengwc.asia ──▸ Cloudflare Tunnel (TCP 7844) ──▸ 后端
受限网络（办公室WiFi）──▸ armbian.tail362392.ts.net ──▸ Tailscale Funnel (TCP 443) ──▸ 后端

网络环境	app.pengwc.asia	armbian.tail362392.ts.net
家庭 WiFi	□	□
手机数据	□	□
办公室 WiFi	□ 端口 7844 被阻断	□ HTTPS 443

---

5. Systemd 开机自启服务

5.1 创建服务文件

/etc/systemd/system/npt-backend.service：

```
[Unit]
Description=Network Planning Tool Backend (FastAPI)
After=network.target
```

```
[Service]
Type=simple
User=root
WorkingDirectory=/DATA/AppData/NetworkPlanningTooV3/backend
ExecStart=/usr/bin/python3 main.py
Restart=on-failure
RestartSec=5
StandardOutput=append:/var/log/npt-backend.log
StandardError=append:/var/log/npt-backend.log

[Install]
WantedBy=multi-user.target
```

5.2 启用服务

```
systemctl daemon-reload
systemctl enable npt-backend.service   # 启用开机自启
systemctl start npt-backend.service    # 立即启动
```

5.3 验证服务

```
systemctl status npt-backend.service
# 预期输出: active (running)

# 查看日志
journalctl -u npt-backend.service -f
# 或
tail -f /var/log/npt-backend.log
```

5.4 开机自启验证

重启后自动验证：

```
# 重启后执行
systemctl is-enabled npt-backend.service   # 预期: enabled
systemctl is-active npt-backend.service    # 预期: active
curl http://127.0.0.1:8000/health          # 预期: {"status":"healthy"}
```

---

6. 域名访问验证

6.1 全面验证清单

```
# 1. 前端首页
curl -s -w "\nHTTP:%{http_code}" https://app.pengwc.asia/ | tail -3
# 应返回 HTML ( <!DOCTYPE html>... )，状态码 200

# 2. SPA 路由
curl -s -o /dev/null -w "%{http_code}" https://app.pengwc.asia/pci
```

```
# 预期: 200
curl -s -o /dev/null -w "%{http_code}" https://app.pengwc.asia/tac/planning
# 预期: 200

# 3. API 端点
curl -s https://app.pengwc.asia/health
# 预期: {"status":"healthy"}

# 4. 静态资源（JS/CSS）
curl -s -o /dev/null -w "%{http_code}" \
  https://app.pengwc.asia/assets/js/index-*.js
# 预期: 200

# 5. JS 文件哈希是否与 dist-renderer 一致
#（确认 Cloudflare 缓存无问题）
```

6.2 多渠道访问

场景	访问地址	预期
外网（家庭/手机，端口 7844 放行）	https://app.pengwc.asia	✅ 完整应用
外网（办公室/企业 WiFi，端口受限）	https://armbian.tail362392.ts.net	✅ 完整应用（Funnel 备用）
内网访问	http://192.168.101.168:8000	✅ 完整应用
本地 localhost	http://127.0.0.1:8000	✅ 完整应用

---

7. 常见问题与故障排除

7.1 前端显示旧版本

症状： 通过域名打开页面，功能仍为旧版。

检查步骤：

```
# 1. 检查构建文件时间
stat /DATA/AppData/NetworkPlanningTooV3/frontend/dist-renderer/assets/js/index-*.js

# 2. 检查源码修改时间
stat /DATA/AppData/NetworkPlanningTooV3/frontend/src/renderer/pages/PCIPage.tsx

# 3. 如果构建文件早于源码 → 需要重新构建
cd /DATA/AppData/NetworkPlanningTooV3/frontend && npm run build:vite
```

根本原因： 构建产物 dist-renderer/ 未更新，仍为旧代码编译结果。

关键经验： 构建文件的JS文件名包含内容哈希（如 index-DJRc6RDA.js），如果源码修改后未重新构建，哈希不变，

7.2 ARM64 架构兼容问题

症状： vite build 报错 Cannot find module @rollup/rollup-linux-arm64-gnu

原因： node_modules 是在 x86 架构上安装的，缺失 ARM64 原生模块。

解决：

```
cd /DATA/AppData/NetworkPlanningTooV3/frontend
rm -rf node_modules package-lock.json  # 仅当完全重建时
npm install  # 自动安装对应架构的原生模块
```

7.3 源码引用但 package.json 未声明的依赖

症状： 构建时提示 Rollup failed to resolve import "xxx"

原因： 旧版 node_modules 中遗留了手动安装的包，但 package.json 中未记录。重新 npm install 后这些包丢失。

排查与修复：

```
# 根据错误提示逐一安装缺失包
npm install i18next react-i18next
npm install react-colorful
npm install terser  # 如果 vite.config.ts 使用 terser 压缩
```

7.4 Cloudflare 隧道连接异常 / 部分网络无法访问

症状 1： 域名在手机/家庭可以访问，办公室/企业 WiFi 无法访问

根因： Cloudflare Tunnel 连接目标端口为 TCP 7844（非标准端口）。企业防火墙通常只放行 80/443 端口，导致 7844 被阻断。

诊断：

```
# 查看 cloudflared 连接日志
journalctl -u cloudflared -n 30 --no-pager | grep -E "Failed|timeout|Registered"

# 如果看到以下错误，说明端口被阻断：
# Failed to dial a quic connection error="... timeout: no recent network activity"
# TLS handshake with edge error: ... i/o timeout
```

解决方案： 1. 切换协议为 HTTP2（TCP 比 QUIC/UDP 更稳定）： bash # 修改 cloudflared.service 中的 ExecStart: # --protocol quic → --protocol http2 # TimeoutStartSec=15 → TimeoutStartSec=30 systemctl daemon-reload && systemctl restart cloudflared

2. 使用 Tailscale Funnel 作为备用通道（在受限网络上使用）：

   https://armbian.tail362392.ts.net/ （走 HTTPS 443，任何防火墙放行）

   配置详见 4.6 节。

症状 2： 隧道进程运行但无法连接

检查步骤：

```
# 1. 检查隧道服务状态
systemctl status cloudflared --no-pager
```

```
# 2. 检查本机后端
curl http://127.0.0.1:8000/health

# 3. 检查 Cloudflare Zero Trust Dashboard 中的隧道状态

# 4. 测试端口 7844 出站连通性
nc -zv -w5 cloudflared.argo.tools 7844
```

7.5 后端占用问题排查

```
# 查端口占用
ss -tlnp | grep 8000

# 强制终止
kill -9 <PID>

# 重启
systemctl restart npt-backend.service

# 看日志
journalctl -u npt-backend.service -n 50 --no-pager
```

7.6 上传文件提示 405 Method Not Allowed（双重 /api/v1 路径）

症状： 在数据管理页面上传工参/地理化文件时，控制台报错 api/v1/api/v1/data/upload/excel:1 Failed to load resource: 405 ()。URL 中出现双重 /api/v1/api/v1/。

根本原因： frontend/src/renderer/services/api.ts 中 uploadExcel 和 uploadGeoData 两个函数在上传 URL 中写死了 /api/v1/data/upload/excel，而 uploadClient 在生产模式下的 baseURL 也是 /api/v1。axios 将两者拼接导致请求地址变为 /api/v1/api/v1/data/upload/excel，后端无此路由返回 405。

修复方法：

frontend/src/renderer/services/api.ts，两处修改：

```
// 第 242 行（uploadExcel）：
// 原代码（错误）：
const uploadUrl = isDev ? `${DEV_BACKEND_URL}/data/upload/excel` : '/api/v1/data/upload/excel'
// 修复后（uploadClient.baseURL 已含 /api/v1，只需 /data/upload/excel）：
const uploadUrl = isDev ? `${DEV_BACKEND_URL}/data/upload/excel` : '/data/upload/excel'

// 第 357 行（uploadGeoData）：
// 原代码（错误）：
const uploadUrl = isDev ? `${DEV_BACKEND_URL}/data/upload/geo` : '/api/v1/data/upload/geo'
// 修复后：
const uploadUrl = isDev ? `${DEV_BACKEND_URL}/data/upload/geo` : '/data/upload/geo'
```

为何开发环境无此问题： - 开发环境 isDev=true，uploadClient.baseURL = DEV_BACKEND_URL = http://host:8000/api/v1 - URL 是完整的绝对路径，axios 不拼接 baseURL - 而生产环境 isDev=false，uploadClient.bas = '/api/v1'（相对路径），axios 会与请求 URL 拼接

验证方法：

```
# 构建后检查产物是否包含旧路径
grep "/api/v1/data/upload" dist-renderer/assets/js/index-*.js
# 如果不返回结果，说明修复生效
```

7.7 electron 依赖安装报错

本项目前端依赖中声明了 electron 和 electron-builder，在 ARM Linux 上安装时会下载 x86 版本失败。如果仅需 Web 模式部署，可以忽略这些错误：

```
# 设置环境变量跳过 electron 下载
export ELECTRON_SKIP_BINARY_DOWNLOAD=1
npm install  # electron 会报错但不影响构建
```

7.8 上传文件提示 524 The origin web server did not return a complete response（Cloudflare 超时）

症状： 上传大文件（如全量工参 Excel，4~5MB）时提示 524 错误：

The origin web server did not return a complete response within the 120-second Proxy Read Timeout window.

根本原因： Cloudflare Tunnel 的上传速度约为 30~35 KB/s。4.5MB 的文件需要约 127 秒才能完成上传，而 Cloudflare 的 proxy read timeout 上限是 100~120 秒。当文件还未完全传到服务器时，Cloudflare 已经超时断开连接。这与后端处理速度无关（即使处理只需 0.2 秒也无济于事）。

验证方法： 对比本地直连与 Cloudflare 域名的上传速度：

```
# 本地直连（应 < 1 秒）
time curl -s -X POST http://127.0.0.1:8000/api/v1/data/upload/excel -F "file=@test_5mb.xlsx"

# 通过 Cloudflare 域名（可能 > 120 秒，返回 524）
time curl -s -X POST https://app.pengwc.asia/api/v1/data/upload/excel -F "file=@test_5mb.xlsx"
```

解决方案：分块上传（Chunked Upload）

将大文件在浏览器端切分为 500KB 的小块，每块单独通过 Cloudflare 上传（每块约 15~17 秒），服务端接收后重组。120 秒限制，彻底避免 524 超时。

后端新增端点（backend/app/api/v1/endpoints/data.py）：

```
# 流程: /upload/start -> /upload/{id}/chunk (N次) -> /upload/{id}/complete

@router.post("/upload/start")
async def start_chunked_upload(filename: str = Form(...), total_chunks: int = Form(...), total_size: int = Form(...)):
    """初始化分块上传会话，返回 data_id"""
    data_id = str(uuid.uuid4())
    chunks_dir = settings.UPLOAD_DIR / f"chunks_{data_id}"
    chunks_dir.mkdir(parents=True, exist_ok=True)
    _chunked_sessions[data_id] = {"filename": safe_filename, "chunks_dir": str(chunks_dir), ...}
    return {"success": True, "data": {"id": data_id, "total_chunks": total_chunks}}

@router.post("/upload/{data_id}/chunk")
async def upload_chunk(data_id: str, chunk_index: int = Form(...), file: UploadFile = File(...)):
    """上传单个分块（500KB），含重试机制"""
    session = _chunked_sessions.get(data_id)
    content = await file.read()
```

```
    chunk_path = Path(session["chunks_dir"]) / f"chunk_{chunk_index:06d}"
    chunk_path.write_bytes(content)
    return {"chunk_index": chunk_index, "size": len(content), "received": len(session["received"]), "total": session["to

@router.post("/upload/{data_id}/complete")
async def complete_chunked_upload(data_id: str, filename: str = Form(...)):
    """重组文件、创建索引、启动后台处理"""
    # 1. 按顺序合并所有分块为完整文件
    # 2. 清理分块临时目录
    # 3. 创建 processing 状态索引
    # 4. asyncio.create_task(_background_process()) 后台解析
    return {"success": True, "data": {"id": data_id, "name": filename, "status": "processing", "fileType": "pending"}}
```

前端修改（frontend/src/renderer/services/api.ts）：

```
// 分块上传参数
const CHUNK_SIZE = 500 * 1024  // 500KB 每块
const MAX_CHUNK_RETRIES = 3    // 每块最多重试 3 次

async function chunkedUpload(file: File, basePath: string): Promise<any> {
  // 阶段1: start - 创建上传会话
  const startResp = await uploadClient.post(`${basePath}/start`, formData)
  const dataId = startResp.data.id

  // 阶段2: 逐块上传（每块 500KB，逐个上传避免超时）
  for (let i = 0; i < totalChunks; i++) {
    const chunk = file.slice(i * CHUNK_SIZE, Math.min((i+1) * CHUNK_SIZE, file.size))
    await uploadClient.post(`${basePath}/${dataId}/chunk`, chunkFormData)
  }

  // 阶段3: complete - 触发服务端重组和后台处理
  const result = await uploadClient.post(`${basePath}/${dataId}/complete`, completeForm)

  // 阶段4: 轮询等待后台处理完成（每 2 秒，最长 5 分钟）
  // ...（与原异步上传逻辑相同）
}

// uploadExcel 和 uploadGeoData 均改为调用 chunkedUpload
uploadExcel: async (file, filePath?) => {
  return await chunkedUpload(file, '/data/upload')
}
```

验证分块上传是否生效：

```
# 检查构建产物中是否包含分块上传逻辑
grep "/data/upload/start\|/data/upload/complete" dist-renderer/assets/js/index-*.js

# 通过 curl 测试分块上传流程
data_id=$(curl -s -X POST https://app.pengwc.asia/api/v1/data/upload/start \
  -F "filename=test.xlsx" -F "total_chunks=3" -F "total_size=3145728" \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['data']['id'])")
```

```
# 逐个上传分块（每个应在 15~30 秒内完成）
for i in 0 1 2; do
  curl -s -X POST "https://app.pengwc.asia/api/v1/data/upload/${data_id}/chunk" \
    -F "file=@chunk${i}.bin" -F "chunk_index=${i}"
done

# 完成上传并启动后台处理
curl -s -X POST "https://app.pengwc.asia/api/v1/data/upload/${data_id}/complete" \
  -F "filename=test.xlsx"
```

---

8. 维护命令速查

8.1 后端管理

```
# systemd 服务管理
systemctl status npt-backend       # 查看状态
systemctl start npt-backend        # 启动
systemctl stop npt-backend         # 停止
systemctl restart npt-backend      # 重启
systemctl enable npt-backend       # 启用开机自启
systemctl disable npt-backend      # 禁用开机自启

# 查看日志
journalctl -u npt-backend -f       # 实时日志
journalctl -u npt-backend -n 100   # 最近 100 条
tail -f /var/log/npt-backend.log   # 日志文件
```

8.2 前端构建

```
cd /DATA/AppData/NetworkPlanningTooV3/frontend

npm install            # 安装/修复依赖
npm run build:vite     # 构建前端
npm run dev:web        # 启动开发服务器（端口 5173）
npm run lint           # 代码检查
```

8.3 Cloudflare Tunnel

```
systemctl status cloudflared --no-pager  # 查看状态
systemctl restart cloudflared            # 重启隧道
journalctl -u cloudflared -f             # 实时日志
journalctl -u cloudflared -n 30          # 最近 30 条

# 手动启动隧道（调试用）
cloudflared tunnel run --token <TOKEN> --protocol http2
```

8.4 Tailscale Funnel 备用通道

```
tailscale serve status          # 查看 serve 配置
tailscale funnel status         # 查看 funnel 状态

# 重新启用（重启后一般无需手动操作，配置自动恢复）
tailscale funnel --bg --set-path / http://localhost:8000

# 关闭 Funnel
tailscale funnel --https=443 off
```

8.5 快速部署流程（从零开始）

```
# 第 1 步：启动后端
systemctl start npt-backend

# 第 2 步：验证后端
curl http://127.0.0.1:8000/health

# 第 3 步：构建前端（源码更新后需要）
cd /DATA/AppData/NetworkPlanningTooV3/frontend
npm run build:vite

# 第 4 步：验证前端
curl http://127.0.0.1:8000/ | head -5

# 第 5 步：验证隧道
curl -s https://app.pengwc.asia/health

# 完成
```

8.6 一键全部重启

```
# 重启所有系统服务
systemctl restart tailscaled cloudflared npt-backend

# 等 tailscaled 就绪后恢复 Funnel（配置在云端，2 秒后生效）
sleep 2 && tailscale funnel --bg --set-path / http://localhost:8000
```

8.7 文件结构参考

```
/DATA/AppData/NetworkPlanningTooV3/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── __init__.py        # ← 含前端静态文件托管配置
│   │   │   └── v1/
│   │   │       ├── __init__.py    # 路由注册
│   │   │       └── endpoints/     # API 端点
│   │   ├── services/              # 业务逻辑
```

```
│   │   ├── algorithms/          # 规划算法
│   │   └── core/config.py       # 配置
│   ├── main.py                  # 启动入口
│   └── requirements.txt
├── frontend/
│   ├── dist-renderer/           # ← 前端构建产物
│   │   ├── index.html
│   │   └── assets/js/ css/
│   ├── src/renderer/
│   │   ├── pages/               # 页面组件
│   │   ├── components/          # UI 组件
│   │   ├── services/api.ts      # API 客户端
│   │   └── store/               # Zustand 状态管理
│   ├── vite.config.ts
│   └── package.json
├── data/                        # 上传的数据文件
├── outputs/                     # 规划结果输出
└── exports/                     # 导出文件
```

附录：完整部署脚本

```bash
#!/bin/bash
# NetworkPlanningTooV3 Linux 部署脚本
# 用法: bash deploy.sh

set -e

PROJECT_DIR="/DATA/AppData/NetworkPlanningTooV3"

echo "===== 1. 停止旧服务 ====="
systemctl stop npt-backend 2>/dev/null || true

echo "===== 2. 构建前端 ====="
cd "$PROJECT_DIR/frontend"
npm install 2>&1 | tail -3
npm run build:vite

echo "===== 3. 启动后端 ====="
systemctl start npt-backend

echo "===== 4. 验证 ====="
sleep 3
curl -s http://127.0.0.1:8000/health
echo ""
curl -s -o /dev/null -w "前端: %{http_code}\n" http://127.0.0.1:8000/
echo ""
curl -s -o /dev/null -w "Cloudflare: %{http_code}\n" https://app.pengwc.asia/health
echo ""
```

```
curl -s -o /dev/null -w "Funnel: %{http_code}\n" https://armbian.tail362392.ts.net/health

echo "===== 部署完成 ====="
```

文档编写日期：2026-05-10
适用系统：Armbian Linux / Ubuntu / Debian (ARM64 / x86_64)